

AI Project Metrics and Interview Answers

Evidence model for resume percentages plus concise behavioral/technical answers from AI_BQ.txt

Important framing: the numbers below should be presented as measured relative improvements against a defined baseline, not universal guarantees. The methods can reach the targets when the baseline is intentionally simple, the evaluation set is fixed, and hardware/model settings are controlled.

How the Resume Metrics Can Be Reached

15% relevance and coherence lift in the SFT project

A 15% lift is credible if measured as a relative improvement on a composite quality score. For example, score each response on relevance, coherence, instruction following, and factual support using a 1-5 rubric. If the base model averages 3.40 and the tuned model averages 3.91, the relative lift is $(3.91 - 3.40) / 3.40 = 15.0\%$. The chosen methods support this because supervised fine-tuning aligns the model to the domain's preferred format, data cleaning removes contradictory training examples, and prompt-template consistency reduces output variance. Human blind review plus LLM-as-judge scoring should be used together so the result is not just an automated metric artifact.

40% GPU-hour reduction in the fine-tuning pipeline

A 40% reduction is plausible when comparing a naive full-parameter or poorly packed SFT run with a PEFT pipeline. LoRA/QLoRA trains a small set of adapter matrices instead of updating all 8B parameters, reducing memory pressure and enabling larger effective batch sizes. Mixed precision, gradient checkpointing, sequence packing, length-based batching, and removal of overlong/duplicate samples reduce wasted tokens. If the initial run took 50 GPU-hours and the optimized run took 30 GPU-hours at comparable validation loss, the reduction is $(50 - 30) / 50 = 40\%$. The key is to report both time saved and fidelity preserved.

30% inference latency reduction in FastAPI Docker deployment

A 30% latency reduction can be reached by comparing an unoptimized serving baseline with an optimized service on the same hardware and prompt set. Improvements come from loading the model once at startup, using quantized weights/adapters, warming the generation path, enforcing max token limits, avoiding repeated tokenizer/model initialization, enabling efficient attention kernels when available, and batching compatible requests. If baseline P50 latency is 2.0 seconds and optimized P50 is 1.4 seconds, the relative reduction is 30%. Report P50 and P95 latency, concurrency, input/output token lengths, GPU type, and container image version.

15% answer-quality lift in the RAG project

The RAG quality lift is credible when baseline RAG uses simple vector top-k retrieval and the improved system adds HyDE, prompt constraints, and better chunking. HyDE often improves semantic retrieval because the hypothetical answer embeds closer to relevant passages than a short user question. Structure-aware chunking preserves context, while prompts that require citations and insufficient-evidence behavior improve faithfulness. If the baseline answer-quality score is 0.68 and the improved score is 0.78, the lift is $(0.78 - 0.68) / 0.68 = 14.7\%$, which can be rounded only if your reporting convention allows it. Track retrieval recall and answer faithfulness separately.

Measurement Table

Claim	Baseline	Improved method	Formula
SFT quality +15%	Base LLaMA 3 responses	SFT LoRA adapter on cleaned 50K examples	$(\text{new score} - \text{base score}) / \text{base score}$

Claim	Baseline	Improved method	Formula
GPU hours -40%	Naive SFT or unoptimized PEFT	QLoRA, packing, batching, mixed precision	$(\text{base hours} - \text{new hours}) / \text{base hours}$
Latency -30%	Cold/repeated loading or unoptimized server	FastAPI startup load, quantization, warmup, limits	$(\text{base latency} - \text{new latency}) / \text{base latency}$
RAG quality +15%	Vector-only top-k baseline	HyDE, structure-aware chunks, prompt/citation rules	$(\text{new score} - \text{base score}) / \text{base score}$

Interview Answers

1. Walk me through your LLM fine-tuning pipeline, end-to-end.

I start by defining the target behavior and building a fixed evaluation set before training. Then I convert the domain data into instruction-style JSONL, profile it for duplicates, length outliers, leakage, and low-quality responses, and standardize the chat template. I run the base LLaMA 3 8B model on the eval set to create the baseline. For training, I use LLaMA-Factory with LoRA/QLoRA, mixed precision, gradient checkpointing, and length-aware batching. I track loss, throughput, GPU hours, and validation behavior. After training, I evaluate with automatic metrics, LLM-as-judge rubric scoring, and blind human review. Finally, I package the adapter, model card, data card, and serve it through FastAPI in Docker with reproducible latency tests.

2. How did you measure the 15% relevance and coherence improvement?

I measured it as a relative improvement over a fixed baseline evaluation set. First, I generated answers from the base model and from the fine-tuned model using the same prompts, decoding settings, and hardware. Then I scored each answer on relevance, coherence, instruction following, and completeness using a 1-5 rubric. I used both LLM-as-judge scoring for scale and blind human review for calibration. The final score was the average across rubric dimensions. The 15% came from the formula: tuned score minus baseline score, divided by baseline score. I also reviewed win/tie/loss examples to make sure the improvement reflected better domain behavior rather than longer or more confident answers.

3. Explain how LoRA and data preprocessing cut GPU hours by 40%.

LoRA reduced the number of trainable parameters by updating small adapter matrices instead of the full 8B model, so the training job used less memory and could run with a better effective batch size. QLoRA or quantized loading also reduced memory pressure. Data preprocessing contributed because I removed duplicate, empty, overlong, and badly formatted samples, which reduced wasted tokens. I also used token-length profiling, sequence packing, and length-aware batching so each GPU step carried more useful training signal. The comparison was made against an earlier unoptimized run at similar validation quality. GPU hours dropped from the baseline run to the optimized run while preserving eval performance.

4. What reduced inference latency by 30% in your FastAPI Docker deployment?

The biggest latency reduction came from serving discipline rather than one trick. I loaded the model and LoRA adapter once during FastAPI startup instead of reinitializing per request, warmed the generation path, pinned model/tokenizer versions, and used quantized weights where quality stayed stable. I also added request validation, max token limits, consistent decoding settings, and timing middleware so I could measure P50 and P95 latency cleanly. In Docker, I removed unnecessary startup work and kept the image deterministic. The 30% reduction was measured against the same prompt set, hardware, concurrency level, and output-token budget.

5. Describe your end-to-end RAG system for document QA.

The system starts with document ingestion, where PDFs, DOCX, or text files are extracted and normalized while preserving source metadata such as page and section. I chunk the text with structure-aware splitting, generate embeddings, and store chunks in a vector database. The baseline retriever uses similarity search, and the improved

version adds HyDE and optionally hybrid retrieval for exact terms. At query time, the system retrieves relevant chunks, builds a grounded prompt, and asks the model to answer with citations and to say when evidence is insufficient. I benchmarked GPT-4o against open-source LLaMA and Mistral models through the same retrieval context, prompt, and evaluation set.

6. How did HyDE and prompt changes produce the 15% answer-quality lift?

The baseline RAG system embedded the user's question directly, which worked for simple questions but sometimes missed passages when the wording differed from the documents. With HyDE, I first generated a hypothetical answer, embedded that answer, and used it to retrieve context. That often moved retrieval closer to the right semantic region. Prompt changes then improved the final answer behavior: the model had to cite sources, avoid unsupported claims, keep answers concise, and explicitly state when the retrieved context was insufficient. I measured the lift on a fixed question set using answer relevance, faithfulness, citation correctness, and retrieval recall.

7. What chunking and embedding strategies did you choose, and why?

I used structure-aware chunking first, splitting by headings and sections before falling back to paragraph or sentence boundaries. My default chunk size was around 700 to 1000 tokens with 100 to 150 tokens of overlap, because document QA needs enough context to preserve definitions and reasoning while avoiding overly broad chunks that dilute retrieval. I stored metadata like source file, page, heading, and chunk ID for citations. For embeddings, I used a strong general embedding model for quality-focused experiments and a smaller embedding model for cost-sensitive runs. I evaluated the choice with retrieval recall@k and answer faithfulness, not just subjective answer quality.

8. Tell me about a significant iteration you made after results looked wrong.

In the RAG project, early results looked fluent but were not consistently grounded. The model answered from prior knowledge when retrieval returned loosely related chunks. I diagnosed this by logging retrieved passages, source IDs, and final answers side by side. The fix was not only prompt wording; I changed chunking to preserve section boundaries, added HyDE for better semantic retrieval, and made the prompt require citations from retrieved context. I also added an insufficient-evidence path. After that, the system produced fewer unsupported answers and the evaluation separated helpfulness from faithfulness, which made the improvement easier to trust.

9. Why are you interested in this role at Company, specifically?

I am interested because the role sits at the intersection I want to work in: LLM systems, agent orchestration, retrieval, evaluation, and production engineering. My background combines computer engineering, ML research, systems projects, and hands-on LLM application work, so I am excited by roles where the model is only one part of the product. I want to build agents that are useful, measurable, and reliable, with clear tool boundaries and strong evaluation. What attracts me to Company specifically is the chance to work on real user workflows at production scale and learn from engineers who care about both model capability and software quality.